would be maintaining all the changes done to your bank balance. These should be the same at all times in every distributed system they are saved in. And every retrieval should give the same balance, irrespective from where it is checked.

Availability

Availability of data at each node means the system always responds to requests. The data may not be the most recent, but getting the response is most important. Also, the responses should not be erroneous. Caching can be used to respond even though the data may be stale. While this may sound contradictory with consistency, in availability getting a response is very important.

Partition tolerance

Tolerance to network partitions means systems remain online even if network problems occur. During power shutdowns or natural disasters, the system may be down partially but is still tolerant to accept and process the user's requests. Ideally, replication across nodes and networks helps here. For example, if I hold an account in the Chennai branch of a bank but there is a power outage in the city, it should not stop me from accessing the money from another or the same location.

CAP guarantees

The CAP theorem guarantees only two properties at a time; we cannot achieve all three together. We need to trade off these properties based on our requirements. Let's see how.

AP: Availability and partition tolerance

Here the application needs to provide high levels of availability. For example:

- You have one master node and two replica nodes.
- Write to the master; you will be successful immediately after write access.
- Asynchronous replication takes place at the backend.
- One of the replicas loses its connection with the master for the time being.
- Now the disconnected replica is contacted; it gives the response but that's stale.
- As per our requirement it should be available; we are available, but not consistent.
- For example, YouTube needs to be highly available.

CP: Consistency and partition tolerance

Here the application needs to be consistent at all times. For instance:

- You have one master node and two replica nodes.
- Write to the master; you are successful only if all the nodes or the maximum nodes get the replica applied successfully.
- Here the write is blocked until it gets the success message.
- The blocking makes the system slow and not available.
- The replica nodes in the system send a heartbeat (ping) to every other node to keep track if other replicas or primary nodes are alive or dead. If no heartbeat is received within N seconds, then that node is marked as inaccessible.
- Yes, we can call it partition tolerance, because if one replica goes down the other can still serve the purpose.
- As the commit happens in the end, the consistency is maintained.
- Each node also maintains the operations logs for the fall back server coming up and getting synced like other nodes.
- As an example, banking systems need to be highly consistent.

CA: Consistency and availability

CA database enables consistency and availability across all nodes. RDBMS databases are CA guaranteed. As these systems are usually in a single system, we can have both consistency and availability.

The CA database can't deliver fault tolerance. Hence, these cannot be used as distributed systems.

To sum up, we have explored the properties of the ACID model that are crucial for determining how to select a database based on each of its parameters. As the ACID model is mostly used in RDMS – SQL databases because of its nature, it needs to be highly consistent. NoSQL, which is distributed in nature, uses the BASE model, which needs to be highly available. The ACID model is usually used in banking applications while BASE is generally used in social network applications. The CAP theorem states that we have to choose any two parameters from availability, consistency and partition tolerance. Now that we are aware of the two major database models and the theorem that governs them, we can choose the database that meets our requirements. END

References

- *https://alxibra.medium.com/isolation-level-in-rails-847edc9e347d*
- *https://www.udemy.com/course/database-engines-crash-course/learn/lecture/28927902?start=0#overview*
- *https://phoenixnap.com/kb/acid-vs-base*

By: Thangaselvi Arichandrapandian

The author works in a leading bank as AVP. She is a perennial learner and loves the quote: "The more I learn, the more I realise how much I don't know."- Albert Einstein